# AI ASSISTED CODING

**SAI SREEYESH** 2303A510h8

**BATCH – 03** 10 – 03 – 2026

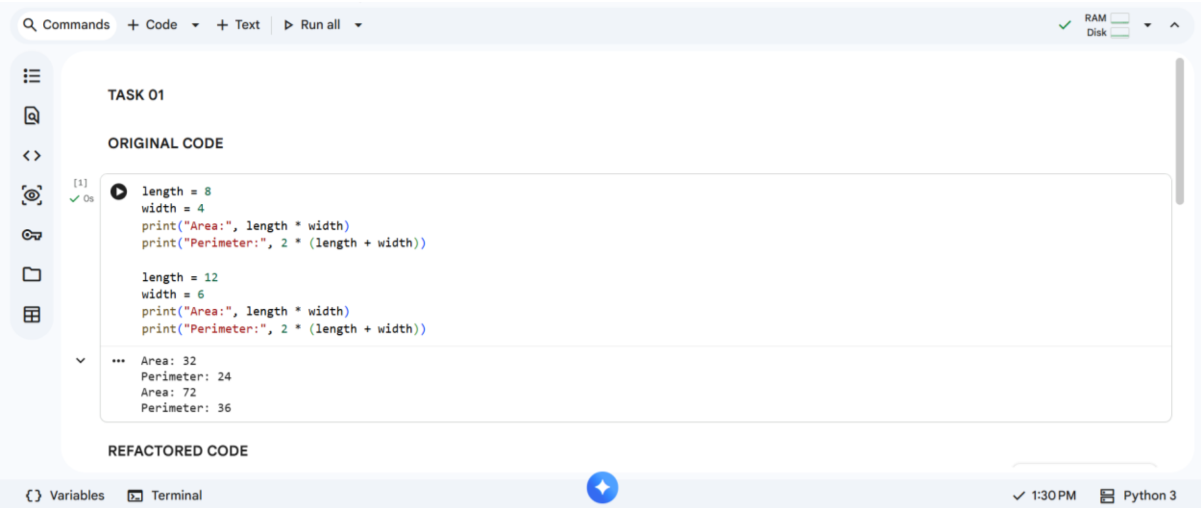
## ASSIGNMENT – 13.2

**LAB – 13 : Code Refactoring – Improving Legacy Code with AI Suggestions.**

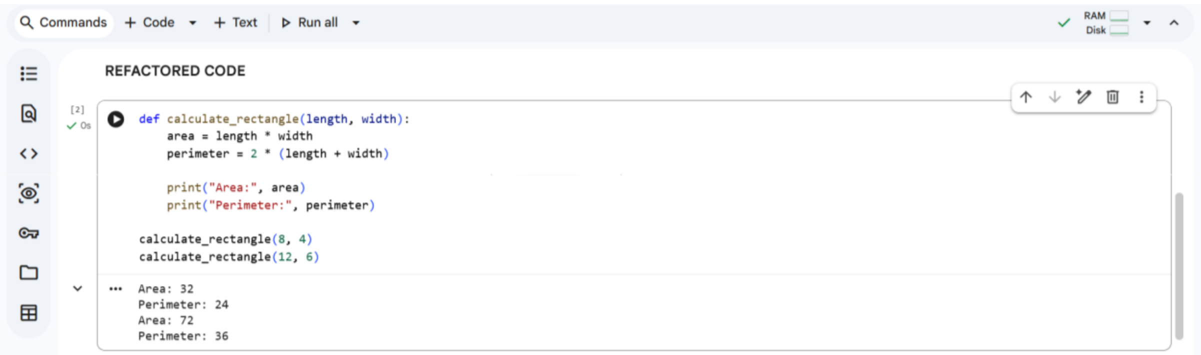
**Task – 01 :** Refactoring – Eliminating Repetitive Logic.

**Prompt :** Use AI to analyze the Python code and identify repeated calculations for area and perimeter. Refactor the code by creating a reusable function with proper docstrings while ensuring the output remains the same.

### Original Code & Output :




### Refactored Code & Output :



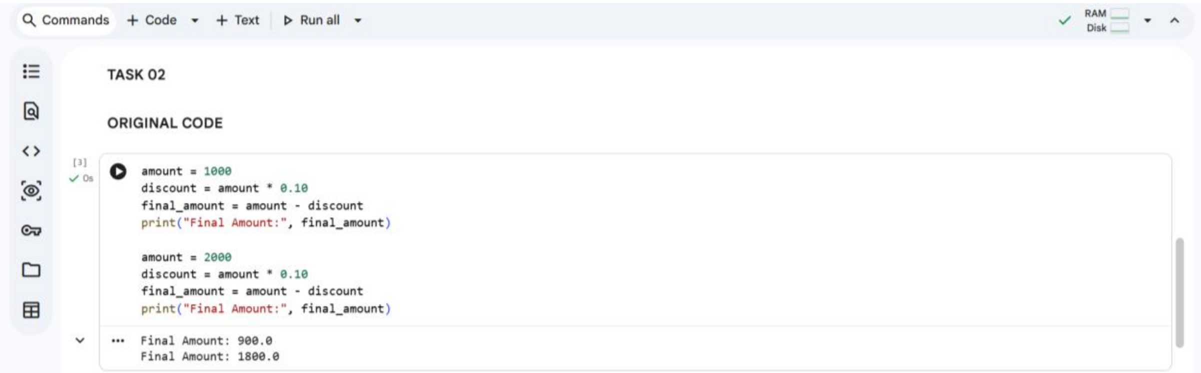
## Explanation :

The repeated calculations for area and perimeter were moved into a reusable function called calculate_rectangle. This improves code readability and reduces duplication. The program still produces the same output but is easier to maintain.

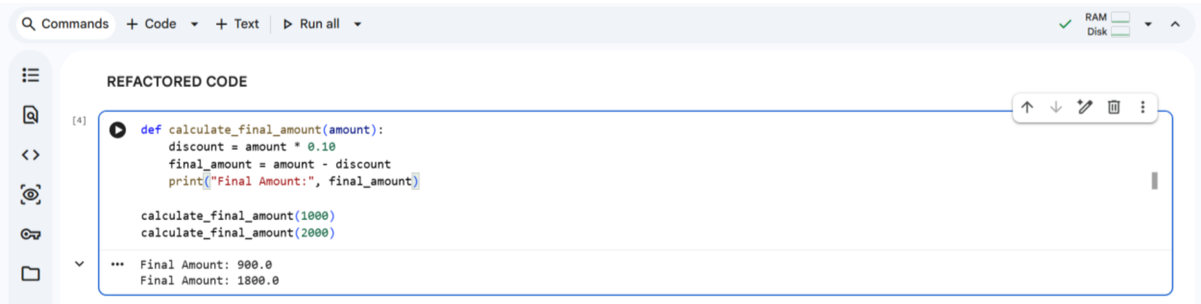
**Task – 02 :** Refactoring – Modularizing Inline Calculations.

**Prompt :** Use AI to refactor the Python code by identifying repeated discount calculations and converting them into a reusable function with proper documentation.

## Original Code & Output :




## Refactored Code & Output :




## Explanation :

The repeated discount calculation logic was moved into a function called calculate_final_amount. This makes the code modular and reusable for different input values. It also improves readability and reduces redundant code.

**Task – 03 :** Refactoring – Improving Loop and Conditional Performance.

**Prompt :** Use AI to optimize the Python program by replacing inefficient nested loops with a faster lookup structure while keeping the output unchanged.

**Original Code & Output :**

TASK 03

ORIGINAL CODE

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
for name in check_list:
  exists = False
  for student in students:
    if name == student:
      exists = True
if exists:
  print(name, "found")
else:
  print(name, "not found")
```

```
Bhavya found
```

**Refactored Code & Output :**

REFACTORED CODE

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

student_set = set(students)

for name in check_list:
    if name in student_set:
        print(name, "found")
    else:
        print(name, "not found")
```

```
Charan found
Esha not found
Bhavya found
```

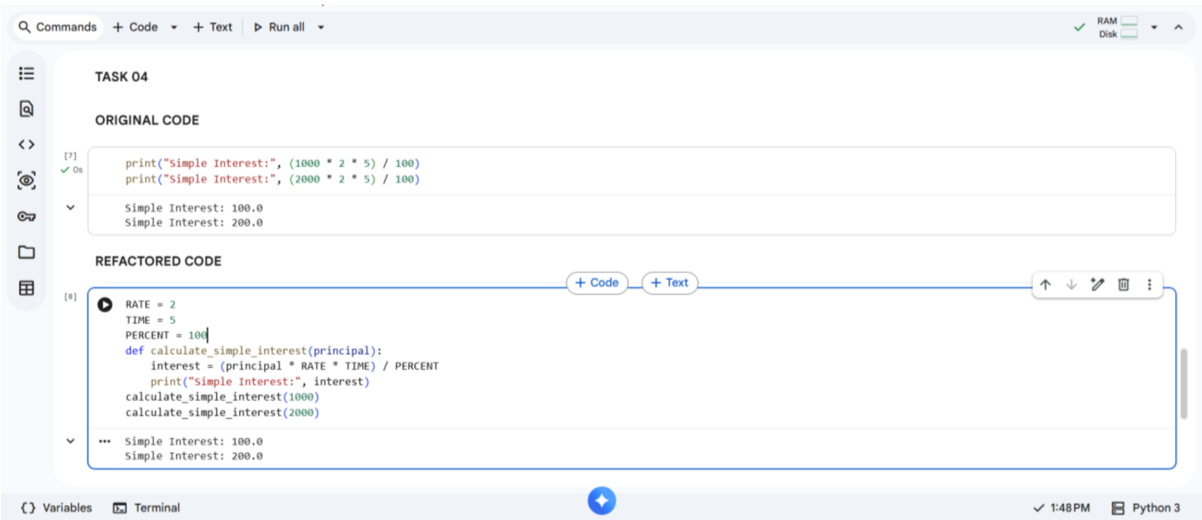
**Explanation :**

The nested loops were replaced with a set data structure, which provides faster lookup operations. Instead of checking each student repeatedly, membership is checked directly using in. This improves performance from O(n²) to O(n).

**Task – 04 :** Refactoring – Replacing Hardcoded Values with Constants.

**Prompt :** Use AI to identify hardcoded numerical values in the simple interest calculation and replace them with descriptive constants to improve maintainability.

**Original/Refactored Code & Output :**



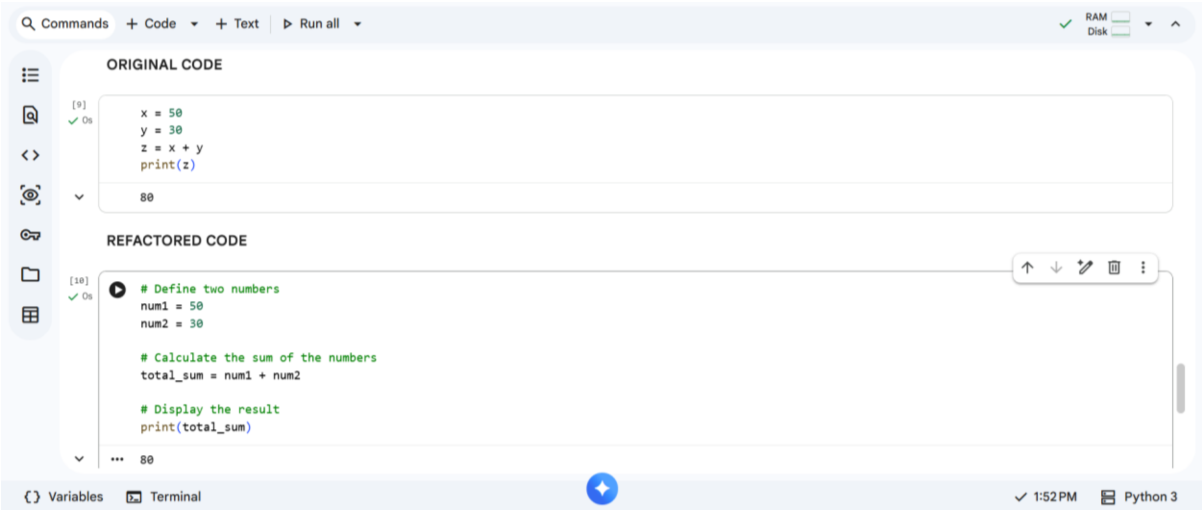

## Explanation :

Hardcoded values such as rate, time, and percentage were replaced with named constants (RATE, TIME, PERCENT). This makes the code easier to understand and modify. If the rate or time changes, it only needs to be updated once.

**Task – 05 :** Refactoring – Enhancing Variable Naming Code Clarity.

**Prompt :** Use AI to improve code readability by replacing unclear variable names with meaningful ones and adding comments explaining the logic.

**Original/Refactored Code & Output :**



**Explanation :**

The variables x, y, and z were renamed to num1, num2, and total_sum to improve clarity. Comments were added to explain each step of the code. This makes the program easier to understand for other developers.

# THANK YOU!!